

PRÁCTICA GUIADA – ETAPA 2

Proyecto “Servicios Técnicos”

Creación y prueba de la API REST de inicio de sesión

Python + Flask + MySQL + JSON

1. Propósito

Construir la primera API REST del proyecto “Servicios Técnicos”. La API recibirá username y contraseña mediante HTTP POST, consultará la tabla usuario de la base de datos tecnico y devolverá una respuesta JSON. En esta etapa todavía no se desarrolla React.

2. Arquitectura

```
React (próxima etapa)
       │ HTTP / JSON
       ▼
Flask → POST /api/login
       │ SQL
       ▼
MySQL → tecnico → usuario
       │
       ▼
Flask → respuesta JSON
```

3. Requisito previo

La Etapa 1 debe estar terminada y la conexión Python → MySQL debe funcionar.

```
cd backend
venv\Scripts\activate
python app/probar_conexion.py
```

Resultado esperado: CONEXIÓN EXITOSA.



4. Verificar estructura

```
Practica_servicios_tecnicos/
├── backend/
│   ├── venv/
│   ├── requirements.txt
│   └── app/
│       ├── __init__.py
│       ├── main.py
│       ├── config/
│       │   ├── __init__.py
│       │   └── conexion.py
│       ├── routes/
│       │   ├── __init__.py
│       │   └── login.py
│       └── services/
├── database/
└── frontend/
```

5. Crear login.py

Abre backend/app/routes/login.py y escribe:

```
from flask import Blueprint, request, jsonify
from app.config.conexion import conectar

login_bp = Blueprint("login", __name__)

@login_bp.route("/api/login", methods=["POST"])
def login():
    datos = request.get_json()

    username = datos.get("username")
    password = datos.get("password")

    if not username or not password:
        return jsonify({
            "success": False,
            "message": "Username y contraseña son obligatorios"
        }), 400

    conexion = conectar()

    if conexion is None:
        return jsonify({
            "success": False,
```




```
            "message": "No se pudo conectar con la base de datos"
        }), 500

    cursor = None

    try:
        cursor = conexion.cursor()

        consulta = """
            SELECT username
            FROM usuario
            WHERE username = %s
            AND password = %s
        """

        valores = (username, password)
        cursor.execute(consulta, valores)
        usuario = cursor.fetchone()

        if usuario:
            return jsonify({
                "success": True,
                "message": "Inicio de sesión correcto",
                "username": usuario[0]
            }), 200

        return jsonify({
            "success": False,
            "message": "Usuario o contraseña incorrectos"
        }), 401

    except Exception as e:
        return jsonify({
            "success": False,
            "message": f"Error en la consulta: {str(e)}"
        }), 500

    finally:
        if cursor:
            cursor.close()
        if conexion:
            conexion.close()
```



6. Explicación

- request.get_json() obtiene los datos enviados por el cliente.
- datos.get() recupera username y password.
- conectar() utiliza la configuración centralizada de conexion.py.
- cursor.execute() ejecuta la consulta SQL.
- fetchone() obtiene el primer registro encontrado.
- jsonify() devuelve la respuesta en formato JSON.
- 200 indica operación exitosa.
- 400 indica datos obligatorios faltantes.
- 401 indica credenciales no válidas.
- 500 indica un error del servidor o conexión.

7. Consulta parametrizada

No concatene directamente los datos del usuario en SQL. Utilice parámetros:

```
consulta = "SELECT username FROM usuario WHERE username = %s AND password = %s"
valores = (username, password)
cursor.execute(consulta, valores)
```

Esta práctica ayuda a evitar inyección SQL y establece una forma segura de enviar los valores a la consulta.

8. Configurar main.py

Abre backend/app/main.py:

```
from flask import Flask, jsonify
from flask_cors import CORS
from app.routes.login import login_bp

app = Flask(__name__)

CORS(app)

app.register_blueprint(login_bp)

@app.route("/")
def inicio():
    return jsonify({
        "mensaje": "API Servicios Técnicos funcionando"
    })
```




```
if __name__ == "__main__":
    app.run(debug=True)
```

9. Ejecutar Flask

Desde backend, con (venv) activo:

```
python -m app.main
```

Debe aparecer:

```
Running on http://127.0.0.1:5000
```

10. Probar GET /

Abre:

```
http://127.0.0.1:5000/
```

Resultado esperado:

```
{ "mensaje": "API Servicios Técnicos funcionando" }
```

11. Probar POST /api/login

En Postman, Insomnia o Thunder Client configura:

```
Método: POST
URL: http://127.0.0.1:5000/api/login
Body → raw → JSON
```

Envía:

```
{ "username": "admin", "password": "12345" }
```

12. Login correcto

```
{ "success": true, "message": "Inicio de sesión correcto", "username": "admin" }
```

Código HTTP esperado: 200.



13. Contraseña incorrecta

```
{ "username": "admin", "password": "99999" }
```

Respuesta esperada:

```
{ "success": false, "message": "Usuario o contraseña incorrectos" }
```

Código HTTP esperado: 401.

14. Campos vacíos

```
{ "username": "", "password": "" }
```

Respuesta esperada:

```
{ "success": false, "message": "Username y contraseña son obligatorios" }
```

Código HTTP esperado: 400.

15. Tabla de pruebas

Prueba	Datos	HTTP	Resultado
Login correcto	admin / 12345	200	☐
Contraseña incorrecta	admin / 99999	401	☐
Usuario inexistente	prueba / 12345	401	☐
Campos vacíos	vacío / vacío	400	☐

16. Flujo de funcionamiento

1. Cliente envía POST /api/login
2. Flask recibe JSON
3. Se extraen username y password
4. Se validan los datos
5. Se conecta a MySQL
6. Se consulta tecnico.usuario
7. Se obtiene el resultado
8. Flask devuelve JSON
9. El cliente recibe la respuesta



17. Solución de problemas

Problema	Qué revisar
No module named 'app'	Ejecutar desde backend: python -m app.main
Import no resuelto	Seleccionar backend\venv como intérprete
404 en /api/login	Revisar URL y registro del Blueprint
405 Method Not Allowed	La ruta requiere POST
401	Revisar datos de tecnico.usuario
500	Revisar conexión y configuración de MySQL

18. Preguntas de análisis

1. ¿Qué es una API REST?

Respuesta: _____

_____

2. ¿Qué es un endpoint?

Respuesta: _____

_____

3. ¿Por qué el login utiliza POST?

Respuesta: _____

_____

4. ¿Qué función cumple jsonify()?

Respuesta: _____

_____

5. ¿Qué significa HTTP 401?

Respuesta: _____

_____

6. ¿Por qué usamos parámetros %s en SQL?

Respuesta: _____



_____

7. ¿Por qué React no consulta directamente MySQL?

Respuesta: _____

_____

19. Registrar la etapa con Git

Cuando todas las pruebas funcionen, detén Flask con Ctrl + C. Desde la raíz del proyecto:

```
git status
git add .
git commit -m "Implementa API de login"
git push
```

Comprueba que el commit aparezca en GitHub.

ANEXO – PRUEBA DE LA API CON THUNDER CLIENT

En esta sección se explica paso a paso cómo instalar Thunder Client en Visual Studio Code, crear una petición HTTP y verificar el endpoint POST /api/login. Esta prueba permite comprobar el backend antes de desarrollar el frontend con React.

A.1. ¿Qué es Thunder Client?

Thunder Client es una extensión de Visual Studio Code que permite realizar peticiones HTTP a APIs. En esta práctica se utilizará para enviar username y password a Flask y observar la respuesta JSON del servidor.

A.2. Verificar si Thunder Client está instalado

1. Abre Visual Studio Code.
2. Observa la barra lateral izquierda.
3. Busca el icono de Thunder Client.
4. Si el icono aparece, haz clic en él y continúa con el apartado A.4.
5. Si no aparece, continúa con el apartado A.3 para instalarlo.



A.3. Instalar la extensión Thunder Client

1. En Visual Studio Code, haz clic en el icono Extensions (Extensiones) de la barra lateral izquierda.
2. En el buscador escribe: Thunder Client.
3. Selecciona la extensión llamada Thunder Client.
4. Haz clic en Install / Instalar.
5. Espera a que termine la instalación.
6. Después de instalarla, debe aparecer el icono de Thunder Client en la barra lateral.

Importante: instala la extensión dentro de Visual Studio Code. No es necesario instalar Postman para realizar esta práctica.

A.4. Abrir Thunder Client

1. Haz clic en el icono de Thunder Client.
2. Selecciona New Request.

Thunder Client puede abrir una solicitud nueva con valores predeterminados. Es normal que aparezca inicialmente GET y una URL de ejemplo. Debemos cambiar ambos valores.

A.5. Configurar el método HTTP

1. En la parte superior de la solicitud localiza el selector que muestra GET.
2. Haz clic en la flecha.
3. Selecciona POST.

Debe quedar:

```
POST
```

A.6. Configurar la URL del endpoint

1. En el campo situado a la derecha de POST borra la URL que aparezca.
2. Escribe exactamente:

```
http://127.0.0.1:5000/api/login
```

También puede utilizarse http://localhost:5000/api/login, pero para esta práctica se recomienda usar 127.0.0.1.



A.7. Configurar el Body como JSON

1. Haz clic en la pestaña Body.
2. Selecciona JSON.
3. En el editor escribe exactamente:

```
{
    "username": "admin",
    "password": "1234"
}
```

Utiliza la contraseña que realmente exista en tu tabla tecnico.usuario. Por ejemplo, si durante la práctica se creó el usuario admin con contraseña 1234, utiliza 1234. Si tu registro utiliza 12345, cambia el valor correspondiente.

A.8. Verificar Content-Type

1. Haz clic en la pestaña Headers.
2. Comprueba que exista el encabezado Content-Type.
3. El valor debe ser:

```
Content-Type: application/json
```

Cuando se selecciona Body → JSON, Thunder Client normalmente agrega este encabezado automáticamente. Si no aparece, agrégalo manualmente.

Header	Value
Content-Type	application/json

A.9. Revisar la petición antes de enviarla

Antes de presionar Send, la configuración debe ser:

```
Método: POST
URL: http://127.0.0.1:5000/api/login
Body: JSON
Content-Type: application/json
```

El JSON de prueba:

```
{
    "username": "admin",
    "password": "1234"
}
```



A.10. Iniciar el servidor Flask

La terminal de Flask debe estar ejecutándose. Si el servidor no está activo, abre una terminal en backend, activa el entorno virtual y ejecuta:

```
cd backend
venv\Scripts\activate
python -m app.main
```

Debe aparecer algo parecido a:

```
Running on http://127.0.0.1:5000
```

No cierres esta terminal durante las pruebas.

A.11. Enviar la petición

1. Regresa a Thunder Client.
2. Verifica POST.
3. Verifica la URL /api/login.
4. Verifica el Body en formato JSON.
5. Verifica Content-Type: application/json.
6. Haz clic en Send.

A.12. Resultado esperado: login correcto

Si el usuario y contraseña existen en la tabla usuario, Thunder Client debe mostrar:

```
Status: 200 OK
```

Y una respuesta similar a:

```
{
    "message": "Inicio de sesión correcto",
    "success": true,
    "username": "admin"
}
```

Además, en la terminal de Flask debe aparecer una petición POST a /api/login con código 200.



A.13. Segunda prueba: contraseña incorrecta

Cambia únicamente la contraseña:

```
{
    "username": "admin",
    "password": "99999"
}
```

Presiona Send.

Resultado esperado:

```
{
    "success": false,
    "message": "Usuario o contraseña incorrectos"
}
```

Código HTTP esperado:

```
401 Unauthorized
```

A.14. Tercera prueba: usuario inexistente

Envía:

```
{
    "username": "usuario_prueba",
    "password": "1234"
}
```

Resultado esperado:

```
{
    "success": false,
    "message": "Usuario o contraseña incorrectos"
}
```

Código HTTP esperado:

```
401 Unauthorized
```



A.15. Cuarta prueba: campos vacíos

Envía:

```
{
    "username": "",
    "password": ""
}
```

Resultado esperado:

```
{
    "success": false,
    "message": "Username y contraseña son obligatorios"
}
```

Código HTTP esperado:

```
400 Bad Request
```

A.16. Quinta prueba: JSON incorrecto

Esta prueba ayuda a comprobar que la API también controla solicitudes mal formadas. Por ejemplo, envía un texto en lugar de un objeto JSON:

```
"username=admin&password=1234"
```

Si se implementó la validación recomendada con request.get_json(silent=True), la API debe responder con un error controlado, no con un traceback.

```
{
    "success": false,
    "message": "El cuerpo de la petición debe ser un objeto JSON"
}
```

Código HTTP esperado: 400.



A.17. Tabla de resultados

Prueba	Username	Password	HTTP esperado	Realizada
Login correcto	admin	1234	200	☐
Contraseña incorrecta	admin	99999	401	☐
Usuario inexistente	usuario_prueba	1234	401	☐
Campos vacíos	vacío	vacío	400	☐
JSON incorrecto	No aplica	No aplica	400	☐

A.18. ¿Cómo interpretar el resultado?

- 200 OK: Flask recibió las credenciales, consultó MySQL y encontró coincidencia.
- 400 Bad Request: la solicitud no contiene los datos requeridos o no tiene el formato esperado.
- 401 Unauthorized: la solicitud llegó correctamente, pero las credenciales no coinciden.
- 500 Internal Server Error: existe un problema interno del backend, por ejemplo una excepción o problema de conexión.

A.19. Si aparece el error AttributeError: 'str' object has no attribute 'get'

Este error indica que la variable datos está llegando como texto y no como diccionario. En login.py utiliza una validación como la siguiente:

```
datos = request.get_json(silent=True)

if not isinstance(datos, dict):
    return jsonify({
        "success": False,
        "message": "El cuerpo de la petición debe ser un objeto JSON"
    }), 400

username = datos.get("username")
password = datos.get("password")
```

Después verifica nuevamente en Thunder Client que Body esté configurado como JSON y que Content-Type sea application/json.

A.20. Si Thunder Client muestra 500

- Revisa la terminal donde está ejecutándose Flask.
- Lee las últimas líneas del traceback.



- Comprueba que MySQL esté iniciado.
- Comprueba usuario, contraseña y base de datos en conexion.py.
- Comprueba que la tabla usuario exista en tecnico.
- Comprueba que login.py importe correctamente conectar().

A.21. Registrar la etapa en GitHub

Desde la raíz del proyecto, después de guardar todos los cambios:

```
git status
git add .
git commit -m "Prueba API de login con Thunder Client"
git push
```

Verifica en GitHub que el commit aparezca. No subas la carpeta venv si el proyecto está configurado para ignorarla mediante. gitignore.